TỔNG QUAN BÀI THI

Học sinh dùng ngôn ngữ lập trình C/C^{++} hoặc Python lập trình các chương trình máy tính đọc file dữ liệu vào, xử lý thông tin rồi ghi kết quả vào file dữ liệu ra.

Bài	File Mã nguồn	File Dữ liệu vào	File Dữ liệu ra	Bộ nhớ	Điểm
Bài 1	BAI1.*	BAI1.INP	BAI1.OUT	1024MB	5
Bài 2	BAI2.*	BAI2.INP	BAI2.OUT	1024MB	5
Bài 3	BAI3.*	BAI3.INP	BAI3.OUT	1024MB	6
Bài 4	BAI4.*	BAI4.INP	BAI4.OUT	1024MB	6
Bài 5	BAI5.*	BAI5.INP	BAI5.OUT	1024MB	8

Chú ý:

*- Dấu * là CPP hoặc PY trong tên file mã nguồn tương ứng với ngôn ngữ C^{++} hoặc Python;*

- Học sinh đặt tên file mã nguồn theo đúng quy định của từng bài, không ghi bất kỳ thông tin cá nhân nào vào file bài làm (họ tên, số báo danh, ngày sinh, trường, các ký hiệu khác thường...);

- File dữ liệu vào và file dữ liệu ra ở trong thư mục hiện hành, thí sinh không phải khai báo đường dẫn đến từng file dữ liệu vào và file dữ liệu ra.

ĐỀ BÀI

BÀI 1. SỐ MAY MẮN (5 điểm) : 20 tests ; 0.25 đ/ test

Một trong những trò chơi thú vị của học sinh thế hệ 8X là tìm ra số may mắn từ ngày tháng năm sinh, số thẻ học sinh, số CMND, số điện thoại... Một học sinh sở hữu con số may mắn khi tổng các chữ số của số đó thoả mãn điều kiện nào đó, ví dụ có tổng các chữ số là 9. Viết chương trình giúp các học sinh này tính tổng các chữ số của mình.

Yêu cầu: Tính tổng các chữ số của số nguyên dương n. ($n \le 10^{19}$)

Dữ liệu: file văn bản BAI1.INP gồm: một số nguyên dương n.

Kết quả: file văn bản BAI1.OUT gồm: một số tổng các chữ số của n.

Ví dụ:

BAI1.INP	BAI1.OUT
23	5

Chú ý:

- Có 60% số test ứng với 60% số điểm thỏa mãn $n \le 10^3$;
- Có 40% số test ứng với 40% số điểm thỏa mãn $n \le 10^{19}$.

BÀI 2. THANH LÝ (5 điểm) : 10 tests ; 0.5 đ /test

Hàng loạt sách cũ của nhà sách cần được thống kê và thanh lý. Tuy nhiên, công việc khó khăn vì mỗi đầu sách có nhiều cuốn có mã số giống nhau, và được để lộn xộn trong kho. Các nhân viên phải tiến hành phân loại bằng cách thống kê từng đầu sách.

Yêu cầu: Hãy xác định xem mỗi đầu sách ***x*** có bao nhiêu cuốn.

Dữ liệu: file văn bản BAI2.INP gồm:

- Dòng đầu tiên chứa số hai số nguyên dương n, m ($n, m \le 10^5$), với n là số cuốn sách trong kho, m là số đầu sách cần phân loại.

- Dòng thứ 2 chứa n số nguyên dương $a_1, a_2, \ldots, a_n$ ($a_i \le 10^9$) là mã số từng cuốn sách.
- m dòng tiếp theo, mỗi dòng chứa một số nguyên ***x*** là mã số đầu sách cần thống kê. ($0 < x \le 10^9$).

Kết quả: file văn bản BAI2.OUT

Ví dụ :

BAI2.INP	BAI2.OUT
5 4 3 4 2 7 3 8 7 3 9	0 1 2 0

Chú ý:

- Có 40% số test tương ứng với 40% số điểm có $n \le 100$;
- Có 60% số test tương ứng với 60% số điểm có $n \le 10^5$.

BÀI 3. GAME (6 điểm) : 12 tests ; 0.5 đ / test

Trong giờ toán, giáo viên tổ chức một trò chơi khởi động. Có *n* đội tham gia giải *m* bài toán khó. Các đội được đánh thứ tự từ 1 đến *n*, cũng chính là tên của đội. Đội thứ *i* giải bài toán *j* hết a_{ij} ($0 < i \le n$, $0 < j \le m$) thời gian, điểm số của mỗi bài giải được là 10. Khi thời gian làm bài kết thúc, nếu bài nào không giải được thì đội không được điểm bài đó. Đội chiến thắng có tổng điểm cao nhất. Hai đội có điểm cao nhất thì đội chiến thắng có tổng thời gian làm bài ít hơn.

Yêu cầu: Viết chương trình tìm ra đội chiến thắng.

Dữ liệu: file văn bản BAI3.INP gồm:

- Dòng 1: chứa 2 số nguyên dương *n* và *m* ($0 < n, m \le 10^3$).
- *n* dòng tiếp theo, mỗi dòng ghi *m* số nguyên a_{ij} (a_{ij} là thời gian giải 1 bài, $0 \le a_{ij} \le 10^3$). Nếu chưa giải được ghi số 0. Các số trên cùng một dòng ghi cách nhau bởi một dấu cách.

Kết quả: file văn bản BAI3.OUT gồm:

- Một dòng ghi tên đội chiến thắng. Nếu có nhiều hơn 1 đội, mỗi đội ghi cách nhau 1 dấu cách theo thứ tự đội từ nhỏ đến lớn.

Chú ý:

- Có 50% số test ứng với 50% điểm của bài toán thỏa mãn $n \le 20$.
- Có 50% test còn lại tương ứng 50% điểm của bài toán không có ràng buộc gì thêm.

Ví dụ:

BAI3.INP	BAI3.OUT
4 5 1 2 4 4 5 3 3 1 1 4 4 0 1 1 3 0 2 1 1 2	2

BÀI 4: PHÁT QUÀ (6 điểm) : 20 tests ; 0.3 đ / test

Một công ty phát quà cho công nhân vào dịp Tết theo **R** nhóm thành tích. Những công nhân đạt cùng thành tích được xếp cạnh nhau. Để tránh mất thời gian, công ty bố trí phương án phát quà như sau: Công nhân của công ty xếp thành một vòng tròn và được đánh số từ 1 đến **N** theo chiều kim đồng hồ. Có **R** lượt phát quà cho công nhân theo quy tắc: mỗi lượt, chỉ phát quà cho công nhân có số thứ tự từ ***x*** đến ***y*** theo chiều kim đồng hồ ($x \le N$, $y \le N$).

Yêu cầu: Cho trước N, R và các cặp số x, y. Tìm số món quà lớn nhất mà công nhân được phát và số lượng công nhân nhận được số quà đó.

Dữ liệu: vào từ tệp văn bản BAI4.INP

- Dòng đầu tiên gồm hai số nguyên dương N và R là số lượng công nhân và số lượt phát quà.
- R dòng sau, mỗi dòng gồm hai số nguyên dương x và y mô tả lượt phát quà.

Kết quả: ghi vào tệp văn bản BAI4.OUT

- Gồm hai số nguyên dương là số quà lớn nhất mà công nhân được phát và số lượng công nhân nhận được số quà như vậy.

Ví dụ:

VÍ DỤ:

BAI4.INP	BAI4.OUT	Giải thích
5 2 1 5 4 2	2 4	Số quà của mỗi người ở mỗi lượt phát quà:
		Ban đầu: 0 0 0 0 0
		Lượt thứ nhất: 1 1 1 1 1
		Lượt thứ hai: 2 2 1 2 2
		Vậy số lượng quà lớn nhất là 2 và có 4 người được 2 quà.

Chú ý:

- 60% số test có 1 <= N, R <= 10^3;
- 20% số test khác có N, R <= 10^5;
- 20% số test còn lại N <= 10^9; R <= 10^5.

BÀI 5. KHẮC PHỤC THIÊN TAI (8 điểm) : 20 tests ; 0.4 đ/test

Một khu phố cần được khắc phục sau thiên tai. Trong khu phố, các hộ dân liền kề nhau cần được hỗ trợ số tiền $a_1, a_2, \ldots, a_n$. Tuy nhiên, tổng ngân sách được cấp tối đa là M nên cần tìm ra số lượng dãy hộ dân liên tiếp có tổng tiền cần hỗ trợ không vượt quá ngân sách trên.

Yêu cầu: Viết chương trình đếm số dãy hộ dân liên tiếp $a_i, a_{i+1}, \ldots, a_j$ ($1 \le i \le j \le n$) sao cho tổng tiền cần hỗ trợ không lớn hơn M.

Dữ liệu: file văn bản BAI5.INP gồm:

- Dòng đầu tiên chứa 2 số nguyên dương n và M; ($M \le 10^9$)
- Dòng thứ 2 chứa n số nguyên $a_1, a_2, \ldots, a_n$. ($1 \le a_i \le 10^6$, tổng các phần tử $a_1, a_2, \ldots, a_n$ nhỏ hơn 10^9)

Kết quả: file văn bản BAI5.OUT gồm: một dòng chứa một số nguyên duy nhất là kết quả tìm được.

Ví dụ 1:

BAI5.INP	BAI5.OUT

6 10 1 2 7 10 15 5	8

Chú ý:

- Có 30% số test ứng với 30% số điểm của bài có $1 \le n \le 10^2$;
- Có 30% số test ứng với 30% số điểm của bài có $10^2 \le n \le 10^3$;
- Có 30% số test ứng với 30% số điểm của bài có $10^3 \le n \le 10^6$.

----------HẾT----------

HƯỚNG DẪN CHẤM

Bài 1.

dùng xâu tính tổng từng chữ số trong xâu

```
for (char x:s)
{s=s+x-‘0’; // hết sức lưu ý phép toán này để biến chữ số thành số
}
```

Bài 2 : Vì n có thể lên đến 10^5, việc duyệt toàn bộ dãy A cho mỗi x sẽ tốn thời gian (O(m*n)). Để tối ưu, chúng ta có thể sử dụng một map hoặc unordered_map để lưu trữ tần suất xuất hiện của mỗi số trong dãy A ngay từ đầu. Điều này sẽ giúp việc tra cứu số lần xuất hiện của x chỉ mất O(log n) hoặc O(1) trung bình.

Bài 3.

Duyệt từng dòng để đếm số bài làm được và tổng thời gian, lưu lại tổng số bài và tổng thời gian để tìm những đội có số bài làm được nhiều nhất với thời gian ít nhất.

Bài 4.

Trên trục số có đoạn thẳng [1, N]. Người ta tiến hành phát quà bởi R đoạn thẳng được giới hạn bởi [X, Y]. Nếu X ≤ Y thì đoạn thẳng [1, N] được phát quà từ X đến Y, nếu X > Y thì đoạn [1, N] được phát quà bởi hai đoạn từ X đến N và từ 1 đến Y.

Yêu cầu: Tìm các đoạn được phát nhiều nhất bao nhiêu lần và tổng độ dài của chúng.

Ý tưởng giải bài toán trên như sau: Với đoạn [X, Y] (X ≤ Y, trường hợp X > Y thì chia thành 2 đoạn [X, N] và [1, Y]): trong đó điểm X được gọi là điểm đầu, Y được gọi là điểm cuối. Trộn tất cả các điểm đầu và cuối vào một mảng seg[] và sắp xếp chúng theo thứ tự tăng dần.

Duyệt các điểm seg[i]:

- Nếu seg[i] là điểm đầu: tăng số đoạn được phát quà count_seg lên 1 đơn vị.
- Nếu seg[i] là điểm cuối: cập nhật số đoạn được phát quà nhiều nhất với count_seg, lưu chúng vào một mảng max[] với số quà được phát nhiều nhất và độ dài của đoạn từ seg[i-1] đến seg[i] sau đó giảm số đoạn được phát quà count_seg xuống 1 đơn vị.

BÀI 5.

Sử dụng thuật toán duyệt chú ý công thức đếm kết quả.

Sử dụng công thức quy hoạch động đơn giản s[i]=s[i-1] + a[i]

Số lượng dãy con thỏa điều kiện kq=i*(i+1)/2

Sử dụng hàm lower_bound